Table des matières

Labo 09 — Ingérer un flux d’événements 1
Avant de commencer — le courtier MQTT, et les clients du chemin chaud 3
Étape 1 — Publier et s’abonner à la main 3
Étape 2 — Faire publier l’usine, et regarder le compresseur 2 4
Étape 3 — Écrire l’abonné 5
Étape 4 — Rejouer une semaine de télémétrie, et l’écouter 11
Étape 5 — Comparer à l’archive 12
Étape 6 — Le message retenu 13
Étape 7 — Couper l’abonné, et voir ce que la session garde 14
Étape 8 — Versionner 15
Avant la seconde séance — Kafka, et le mot de passe de la base 15
Étape 9 — Créer les sujets, et choisir leurs partitions 16
Étape 10 — Écrire le pont 16
Étape 11 — Écrire le consommateur, et les tables qui dédoublonnent 19
Étape 12 — Écrire les fenêtres et l’alerte, et les tester sans Kafka 23
Étape 13 — Lancer toute la chaîne sur octobre 29
Étape 14 — Lire l’alerte dans la base, et son avance 30
Étape 15 — Relire le flux par un second groupe 31
Étape 16 — Faire passer la télémétrie, et compter ses retards 32
Étape 17 — Versionner 32
Récapitulatif 33
Dépannage 33

Labo 09 — Ingérer un flux d’événements

Labo · Cours 9/12 — Formation Data Engineering

© 2026 iAlexMG. Tous droits réservés.

Ce que vous allez faire. Faire publier l'usine et les véhicules de Boréal sur un courtier MQTT, les écouter, et prouver que l'archive de la télémétrie que vous chargez depuis le Cours 03 est ce qu'un abonné écrit. Puis garder le flux dans Kafka, le relire à volonté, en remplir la base de séries temporelles, et en tirer l'alerte de vibration du compresseur 2. Objectif de fin : l'alerte levée le 22 octobre 2025 à 8 h 15, six jours et deux heures et demie avant le déclenchement, et un chemin chaud qui ne perd rien et ne double rien.

Le cours compte deux séances. La première va jusqu'à l'étape 8 et tient MQTT seul ; elle se tient seule. La seconde allume Kafka et va jusqu'au bout.

Trois terminaux servent, et l'un d'eux en plusieurs exemplaires. L'Anaconda Prompt, environnement `formations` actif, lance Python : le rejoueur, depuis `Data_Domain`, et les programmes du chemin chaud, depuis `Data_Domain\Projet_Boreal_intermediaire\01_de`. Un programme en continu occupe son terminal tant qu'il tourne : vous en ouvrirez deux à la première séance et quatre à la seconde, un par programme. Ubuntu, dans WSL 2, sert à Docker, depuis `Projet_Boreal_intermediaire/plateforme`. La ligne de commande DuckDB, lancée depuis `Data_Domain\Projet_Boreal_intermediaire`, sert aux comparaisons avec les fichiers.

La liste de contrôle :

- ☐ Le courtier démarré : `mosquitto version 2.1.2 running`
- ☐ Un message publié à la main et reçu par un abonné, puis un message retenu
- ☐ Le rejeu d'une journée de l'usine : 4 048 messages en 24 s, dont 230 pour le compresseur 2
- ☐ `flux/commun.py` et `flux/abonne.py` écrits
- ☐ La télémétrie reçue : 3 449 messages, 70 doublons, 246 en retard de plus d'une heure
- ☐ L'archive et l'abonné identiques : 3 449 lignes de part et d'autre, 0 écart dans les deux sens
- ☐ Les 58 messages retenus reçus à l'abonnement, et les 4 048 points égaux à l'historien
- ☐ Une coupure de l'abonné, sans une ligne perdue, puis la session fermée
- ☐ Kafka démarré, et ses trois sujets créés
- ☐ Le pont, le consommateur, les fenêtres et l'alerte écrits ; `pytest` rend 8 verts
- ☐ Octobre rejoué de bout en bout : 62 726 points relayés et insérés, l'alerte le 22 à 8 h 15
- ☐ L'avance mesurée : 6 jours 2 h 29 min pour l'alerte, 1 jour 4 h 29 min pour l'arrêt
- ☐ La relecture par un second groupe : levée le 21 octobre à 6 h 45 au seuil de 4,0
- ☐ La télémétrie par Kafka : 3 449 lus, 3 379 insérés, 70 doublons écartés par la base

© 2026 iAlexMG. Tous droits réservés.

Avant de commencer — le courtier MQTT, et les clients du chemin chaud

Ce cours allume deux services de la plateforme, le courtier MQTT à la première séance et Kafka à la seconde. Tous deux sont décrits dans le `docker-compose.yml` de `_univers/plateforme/`, et le courtier demande en plus un fichier de configuration, `mosquitto.conf`. Recopiez les deux, même si vous avez reçu la plateforme au Cours 04 : le fichier de configuration est neuf. Dans l'Anaconda Prompt, depuis `Data_Domain\Projet_Boreal_intermediaire` :

```
copy ..\_univers\plateforme\docker-compose.yml plateforme\
copy ..\_univers\plateforme\mosquitto.conf plateforme\
```

Le code de ce cours parle au courtier, à Kafka et à la base de séries temporelles par trois paquets Python que le chemin froid n'emploie jamais : `paho-mqtt`, `confluent-kafka` et `psycopg`. Ils sont dans le `requirements.txt` de la formation. Vérifiez qu'ils sont là, et installez-les sinon :

```
python -c "import paho.mqtt, confluent_kafka, psycopg; print('ok')"
pip install -r ..\01_Data_Engineering\requirements.txt
```

Démarrez maintenant le courtier. Dans Ubuntu, depuis `plateforme` :

```
docker compose up -d mqtt
docker compose logs mqtt
```

```
boreal_mqtt  | 1790146894: Info: running mosquitto as user: mosquitto.
boreal_mqtt  | 1790146894: mosquitto version 2.1.2 starting
boreal_mqtt  | 1790146894: Config loaded from /mosquitto/config/mosquitto.conf.
boreal_mqtt  | 1790146894: Bridge support available.
boreal_mqtt  | 1790146894: Persistence support available.
boreal_mqtt  | 1790146894: TLS support available.
boreal_mqtt  | 1790146894: TLS-PSK support available.
boreal_mqtt  | 1790146894: Websockets support available.
boreal_mqtt  | 1790146894: Opening ipv4 listen socket on port 1883.
boreal_mqtt  | 1790146894: Opening ipv6 listen socket on port 1883.
boreal_mqtt  | 1790146894: mosquitto version 2.1.2 running
```

La troisième ligne dit que le courtier a bien lu `mosquitto.conf` ; sans elle, il n'écouterait que l'intérieur de son conteneur. Le nombre en tête de chaque ligne est l'heure en secondes depuis 1970, et le vôtre sera autre. L'image pèse 41,8 Mo. Gardez ce terminal Ubuntu ouvert pendant toute la séance : sous Windows 10, WSL arrête Ubuntu, et ses conteneurs avec lui, quelques secondes après la fermeture du dernier terminal.

Étape 1 — Publier et s'abonner à la main

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_09_DE.md`, sections 2 et 3.

Avant d'écrire une ligne de Python, parlez au courtier avec les deux outils qu'il fournit : `mosquitto_sub`, qui s'abonne et affiche, et `mosquitto_pub`, qui publie. Ouvrez un second terminal Ubuntu, placez-vous aussi dans `plateforme`, et abonnez-vous à tout ce qui commence par `boreal/essai/` :

```
docker compose exec mqtt mosquitto_sub -t 'boreal/essai/#' -v
```

Le terminal attend. Dans le premier, publiez deux messages, le second marqué retenu par `-r` :

```
docker compose exec mqtt mosquitto_pub -t boreal/essai/salut -m 'premier message'
docker compose exec mqtt mosquitto_pub -t boreal/essai/etat -r -m 'en marche'
```

L'abonné affiche chaque message, précédé de son sujet grâce à `-v` :

```
boreal/essai/salut premier message
boreal/essai/etat en marche
```

Arrêtez l'abonné par Ctrl+C, et relancez exactement la même commande. Il n'affiche plus qu'une ligne, `boreal/essai/etat en marche` : le premier message est perdu pour qui n'écoutait pas, le second était retenu. Arrêtez-le de nouveau, puis effacez le message retenu en publiant un message vide sur son sujet, par `-n` :

```
docker compose exec mqtt mosquitto_pub -t boreal/essai/etat -r -n
```

Un troisième abonnement n'affiche plus rien. Essayez enfin le joker d'un seul niveau : abonnez-vous à `'boreal/+/salut'`, puis republiez sur `boreal/essai/salut`. Le message arrive, parce que `+` remplace exactement le niveau `essai`. Les apostrophes autour d'un sujet ne sont pas toujours nécessaires, mais elles évitent qu'un terminal interprète un caractère spécial : prenez l'habitude.

Étape 2 — Faire publier l'usine, et regarder le compresseur 2

Théorie : `Theorie_09_DE.md`, section 3.

Aucun automate ne publie sur votre poste. Le rejoueur, `_univers/rejoueurs/rejouer.py`, en tient lieu : il relit ce que le générateur a produit et le publie sur le courtier à l'instant où cela s'est produit, en temps accéléré. Pour l'usine, il publie chaque point de l'historien, valeurs de départ exclues, sur un sujet par étiquette : `UTL.CMP2.VIBRATION` devient `boreal/usine/UTL/CMP2/VIBRATION`. Il ramène l'heure locale de l'historien à un instant absolu, par le recul de l'horloge du Cours 08, et marque chaque point comme retenu. Lisez son en-tête avant de le lancer ; vous ne le modifierez pas, il appartient aux générateurs.

Dans votre second terminal Ubuntu, abonnez-vous aux quatre points du compresseur 2 :

```
docker compose exec mqtt mosquitto_sub -t 'boreal/usine/UTL/CMP2/#' -v
```

Puis, dans l'Anaconda Prompt, depuis `Data_Domain`, rejouez la journée du déclenchement, le 28 octobre 2025. Avec la vitesse par défaut, 3 600, une heure de Boréal passe en une seconde :

© 2026 iAlexMG. Tous droits réservés.

```
python _univers/rejoueurs/rejouer.py usine --projet Projet_Boreal_intermediaire --depuis 2025-10-28 --jusqua 2025-10-28

Rejeu de usine, du 2025-10-28 au 2025-10-28, à la vitesse 3600 : environ 24 s.
  2025-10-28 : 4048 messages
4048 messages publiés en 24 s.
```

Pendant ces 24 secondes, l'abonné affiche 230 lignes, le compresseur 2 vu comme la salle de contrôle le voit. Retrouvez le déclenchement de 10 h 44 :

```
boreal/usine/UTL/CMP2/VIBRATION {"etiquette":"UTL.CMP2.VIBRATION","instant":"2025-10-28T10:40:00-
04:00","valeur":8.94,"qualite":192}
boreal/usine/UTL/CMP2/MARCHE {"etiquette":"UTL.CMP2.MARCHE","instant":"2025-10-28T10:44:00-04:00","valeur":0,"qualite":192}
boreal/usine/UTL/CMP2/PUISSANCE {"etiquette":"UTL.CMP2.PUISSANCE","instant":"2025-10-28T10:44:00-
04:00","valeur":0.0,"qualite":192}
boreal/usine/UTL/CMP2/TEMP_REFOULEMENT {"etiquette":"UTL.CMP2.TEMP_REFOULEMENT","instant":"2025-10-28T10:44:00-
04:00","valeur":96.82,"qualite":192}
boreal/usine/UTL/CMP2/VIBRATION {"etiquette":"UTL.CMP2.VIBRATION","instant":"2025-10-28T10:44:00-
04:00","valeur":0.0,"qualite":192}
```

La vibration est à 8,94 mm/s à 10 h 40 ; à 10 h 44, la marche tombe à 0, la puissance aussi, et la température de refoulement commence à redescendre. Arrêtez l'abonné par Ctrl+C.

Le rejoueur a publié autre chose que les points : un message retenu sur `boreal/rejoueur/usine/horloge`, qui dit à quel instant de Boréal le rejeu a commencé, à quelle heure réelle, et à quelle vitesse. C'est par lui que vos programmes sauront quelle heure il est chez Boréal pendant un rejeu.

Étape 3 — Écrire l'abonné

Théorie : `Theorie_09_DE.md`, sections 4 et 6.

Le chemin chaud vit dans un paquet à part du dépôt, `flux`, à côté de `pipeline` : ses programmes ne finissent pas, et ne se lancent donc pas comme des étapes. Il réemploie la configuration du pipeline. Ouvrez `config.toml`, dans `Projet_Boreal_intermediaire\01_de`, et ajoutez une ligne à la section `[chemins]` :

```toml
# Ce que l'abonné du Cours 09 écrit : ses propres fichiers, à comparer à l'archive de la télémétrie.
flux = "flux"
```

Puis une section neuve, à la fin du fichier :

```toml
[flux]
# Le chemin chaud (Cours 09) : le courtier MQTT, Kafka et la base de séries temporelles, tous trois
# dans la plateforme. Le mot de passe de la base est dans .env, sous SERIES_MOT_DE_PASSE.
courtier = "localhost:1883"
kafka = "localhost:9092"
series_serveur = "localhost"
series_port = 5434
```


© 2026 iAlexMG. Tous droits réservés.

```
series_base = "usine"
series_utilisateur = "postgres"
```

Créez le paquet, depuis l'Anaconda Prompt, dans `01_de` :

```
mkdir flux
notepad flux\__init__.py
```

Ce code ne se tape pas dans la ligne de commande DuckDB : ce sont des fichiers du dépôt. Au moment d'enregistrer, choisissez « Tous les fichiers » dans la liste des types, sans quoi le Bloc-notes ajoute `.txt` au nom. Le premier fichier ne tient qu'une ligne, qui fait du dossier un paquet :

```
# -*- coding: utf-8 -*-
"""flux : le chemin chaud de Boréal, des programmes qui écoutent et ne finissent pas."""
```

Le deuxième, `notepad flux\commun.py`, porte ce que l'abonné et le pont du Cours 09 partagent. Son début : la configuration, la liste des sujets écoutés, et la mise en forme d'un instant à la façon de l'archive.

```
# -*- coding: utf-8 -*-
"""commun.py : la configuration, l'heure de Boréal et la réception, partagées par l'abonné et
le pont."""
import json
import time
from collections import Counter
from datetime import datetime, timedelta
from zoneinfo import ZoneInfo

from pipeline.config import chemin, config

# Sous Windows, zoneinfo lit sa base de fuseaux dans le paquet tzdata, que psycopg installe.
EST = ZoneInfo("America/Toronto")

# Les sujets qu'écoutent l'abonné et le pont. L'horloge en qualité de service 0 : un message retenu
# se renvoie à chaque abonnement, il n'a pas besoin d'accusé, et il ne fait pas la queue derrière
# les messages de qualité 1, que le courtier limite à 20 en vol par client.
SUJETS = [("boreal/rejoueur/+/horloge", 0), ("boreal/usine/#", 1),
          ("boreal/vehicules/+/sorties", 1)]


def reglages() -> dict:
    """La section [flux] de config.toml, surcharges de l'environnement comprises."""
    return config()["flux"]
```


© 2026 iAlexMG. Tous droits réservés.

```
def hote_port(adresse: str) -> tuple[str, int]:
    """« localhost:1883 » devient ("localhost", 1883)."""
    hote, port = adresse.rsplit(":", 1)
    return hote, int(port)


def dossier_flux():
    """Le dossier où l'abonné écrit, Projet_Boreal_intermediaire/flux par défaut."""
    return chemin("flux")


def iso(instant: datetime) -> str:
    """Un instant en heure de l'Est, au format de l'archive : 2026-01-15T00:28:31-05:00."""
    return instant.astimezone(EST).isoformat(timespec="seconds")
```

Notez la qualité de service 0 de l'horloge, au milieu de la liste des sujets : un message retenu se renvoie à chaque abonnement, et il n'a donc pas besoin d'accusé. La suite du même fichier tient l'horloge, et le début du récepteur que l'abonné et le pont partageront :

```
class Horloge:
    """L'heure de Boréal pendant un rejeu, tirée du message retenu du rejoueur."""

    def __init__(self, charge: dict):
        self.debut_simule = datetime.fromisoformat(charge["debut_simule"])
        self.debut_reel = float(charge["debut_reel"])
        self.vitesse = float(charge["vitesse"])

    def maintenant(self) -> datetime:
        """L'instant de Boréal qui correspond à l'heure réelle du poste, à la seconde."""
        ecoule = (time.time() - self.debut_reel) * self.vitesse
        return (self.debut_simule + timedelta(seconds=ecoule)).replace(microsecond=0)


class Recepteur:
    """Reconnaît le flux d'un message, écarte les retenus, le date en heure de Boréal ; l'abonné
    et le pont en héritent, et ne définissent que traiter()."""

    manuel = False

    def __init__(self):
        self.horloges = {}
        self.en_attente = {}
        self.compte = Counter()
        self.dernier = None
```


© 2026 iAlexMG. Tous droits réservés.

```
        self.client = None

    def accuser(self, msg):
        if self.manuel and msg.qos == 1:
            self.client.ack(msg.mid, msg.qos)
```

`Horloge.maintenant()` convertit l'heure du poste en heure de Boréal, par le message retenu du rejoueur. Le reste du récepteur reconnaît chaque message :

```
    def recevoir(self, client, userdata, msg):
        self.client = client
        parties = msg.topic.split("/")
        if parties[1] == "rejoueur":
            flux = parties[2]
            self.horloges[flux] = Horloge(json.loads(msg.payload))
            for attente in self.en_attente.pop(flux, []):
                self.traiter(flux, attente, self.horloges[flux].maintenant())
            return
        flux = "usine" if parties[1] == "usine" else "telemetrie"
        if msg.retain:
            self.compte[f"{flux} : messages retenus reçus à l'abonnement"] += 1
            self.accuser(msg)
            return
        self.dernier = time.time()          # seul un message neuf fait courir le silence
        if flux not in self.horloges:
            self.en_attente.setdefault(flux, []).append(msg)
        else:
            self.traiter(flux, msg, self.horloges[flux].maintenant())

    def traiter(self, flux, msg, recu):
        raise NotImplementedError

    def attendre(self, secondes):
        """Ce que fait le programme entre deux coups d'œil au silence : rien, sauf pour le pont."""
        time.sleep(secondes)

    def terminer(self):
        """Ce qui reste à faire avant de se déconnecter : rien, sauf pour le pont."""
```

Trois cas dans `recevoir()`. L'horloge du rejoueur se range. Un message retenu se compte et s'écarte, parce qu'il est une valeur déjà connue et non une réception. Et un message qui arrive avant l'horloge de son flux attend dans `en_attente` : au retour d'une session persistante, le courtier livre d'abord les messages qu'il a gardés, et seulement ensuite le message retenu de l'horloge. Les jeter serait les perdre, puisque paho les acquitte au retour de la fonction.

© 2026 iAlexMG. Tous droits réservés.

Enfin, seul un message neuf fait courir le compteur de silence : un programme lancé avant un rejeu reçoit dès l'abonnement les messages retenus du rejeu précédent, et s'arrêterait sinon avant d'avoir rien reçu de neuf. La fin du fichier connecte un récepteur au courtier et l'écoute :

```
  def ecouter(recepteur, client_id, persistante, silence):
      """Connecte un récepteur au courtier et l'écoute jusqu'au silence, ou jusqu'à Ctrl+C."""
      import paho.mqtt.client as mqtt                  # seuls l'abonné et le pont en ont besoin

      def connecte(client, userdata, drapeaux, code, proprietes):
          print(f"Connecté ; session reprise : {'oui' if drapeaux.session_present else 'non'}.")
          client.subscribe(SUJETS)

      client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, client_id=client_id,
                           clean_session=not persistante, protocol=mqtt.MQTTv311,
                           manual_ack=recepteur.manuel)
      client.on_connect = connecte
      client.on_message = recepteur.recevoir
      hote, port = hote_port(reglages()["courtier"])
      client.connect(hote, port)
      client.loop_start()                              # un fil de paho reçoit, celui-ci surveille
      print(f"À l'écoute de {hote}:{port}. Ctrl+C pour arrêter.", flush=True)
      try:
          while recepteur.dernier is None or time.time() - recepteur.dernier < silence:
              recepteur.attendre(0.5)
          print(f"Aucun message depuis {silence:g} s : arrêt.")
      except KeyboardInterrupt:
          print("Arrêt demandé.")
      recepteur.terminer()
      client.disconnect()                              # le fil de paho envoie ce qui reste, puis part
      client.loop_stop()
```

L'abonnement se fait dans `connecte()`, appelée à chaque connexion : une reconnexion retrouve ses sujets. Le programme s'arrête seul après un silence, ou par Ctrl+C.

Reste l'abonné lui-même, `notepad flux\abonne.py`. D'abord la classe, qui écrit :

```
  # -*- coding: utf-8 -*-
  """abonne.py : écoute le courtier MQTT et écrit ce qu'il reçoit, au format de l'archive.

  Usage, depuis 01_de : python -m flux.abonne [--silence 15] [--session-propre]
  """
  import argparse
  import json
  from datetime import datetime, timedelta
```


© 2026 iAlexMG. Tous droits réservés.

```
from flux.commun import EST, Recepteur, dossier_flux, ecouter, iso

RETARD = timedelta(hours=1)


class Abonne(Recepteur):
    def __init__(self, dossier):
        super().__init__()
        self.dossier = dossier
        self.vus = set()
        self.fichiers = {}

    def ecrire(self, nom, ligne):
        if nom not in self.fichiers:
            chemin = self.dossier / nom
            chemin.parent.mkdir(parents=True, exist_ok=True)
            # buffering=1 : chaque ligne part sur le disque avant que le message soit acquitté, le
            # courtier ne recevant son accusé qu'au retour de recevoir(). Un abonné tué en plein
            # travail ne perd rien de ce qu'il a acquitté.
            self.fichiers[nom] = open(chemin, "a", encoding="utf-8", newline="\n", buffering=1)
        texte = json.dumps(ligne, ensure_ascii=False, separators=(",", ":"))
        self.fichiers[nom].write(texte + "\n")
```

`buffering=1` fait partir chaque ligne sur le disque dès qu'elle est écrite. Le courtier ne recevant son accusé qu'au retour de la fonction, un abonné tué en plein travail ne perd rien de ce qu'il a acquitté. Puis ce qu'il fait de chaque message, et sa porte d'entrée :

```
    def traiter(self, flux, msg, recu):
        jour = recu.astimezone(EST).date().isoformat()
        charge = json.loads(msg.payload)
        self.compte[f"{flux} : messages reçus"] += 1
        if flux == "telemetrie":
            if charge["message_id"] in self.vus:
                self.compte["telemetrie : doublons"] += 1
            elif recu - datetime.fromisoformat(charge["fin"]) > RETARD:
                self.compte["telemetrie : reçus plus d'une heure après la fin"] += 1
            self.vus.add(charge["message_id"])
            ligne = {"sujet": msg.topic, "recu_le": iso(recu), "qos": msg.qos, "message": charge}
            self.ecrire(f"telemetrie/sorties_{jour}.jsonl", ligne)
        else:
            if charge["valeur"] is None:
                self.compte["usine : points sans valeur"] += 1
            self.ecrire(f"usine/points_{jour}.jsonl", dict(charge, recu_le=iso(recu)))
```


© 2026 iAlexMG. Tous droits réservés.

```
def principal():
    p = argparse.ArgumentParser(description="Écoute le courtier MQTT et écrit ce qu'il reçoit.")
    p.add_argument("--silence", type=float, default=15,
                   help="s'arrêter après tant de secondes sans message")
    p.add_argument("--session-propre", action="store_true",
                   help="ne rien garder chez le courtier en partant")
    a = p.parse_args()

    abonne = Abonne(dossier_flux())
    print(f"Écriture dans {abonne.dossier}.")
    ecouter(abonne, "abonne-01de", persistante=not a.session_propre, silence=a.silence)
    for f in abonne.fichiers.values():
        f.close()
    for cle in sorted(abonne.compte):
        print(f"  {cle} : {abonne.compte[cle]}")


if __name__ == "__main__":
    principal()
```

Pour la télémétrie, l'abonné compte les doublons par `message_id`, et les messages reçus plus d'une heure après la fin de leur sortie. Il écrit chaque ligne dans le format exact de l'archive, `sujet`, `recu_le`, `qos` et `message`, dans un fichier par jour de réception.

Étape 4 — Rejouer une semaine de télémétrie, et l'écouter

Théorie : `Theorie_09_DE.md`, sections 4 et 6.

Lancez l'abonné dans un Anaconda Prompt, depuis `01_de` :

```
python -m flux.abonne
```

```
Écriture dans C:\...\Data_Domain\Projet_Boreal_intermediaire\flux.
À l'écoute de localhost:1883. Ctrl+C pour arrêter.
Connecté ; session reprise : non.
```

Il attend. Ouvrez un second Anaconda Prompt, placez-vous dans `Data_Domain`, et rejouez la semaine du samedi 10 au vendredi 16 janvier 2026, en plein hiver, quand les véhicules sortent souvent de la couverture cellulaire :

```
python _univers/rejoueurs/rejouer.py telemetrie --projet Projet_Boreal_intermediaire --depuis 2026-01-10 --jusqua 2026-01-16
```


© 2026 iAlexMG. Tous droits réservés.

```
Rejeu de telemetrie, du 2026-01-10 au 2026-01-16, à la vitesse 3600 : environ 168 s.
  2026-01-10 : 1094 messages
  2026-01-11 : 1024 messages
  2026-01-12 : 219 messages
  2026-01-13 : 281 messages
  2026-01-14 : 222 messages
  2026-01-15 : 187 messages
  2026-01-16 : 422 messages
3449 messages publiés en 168 s, dont 70 doublons d'un message déjà publié.
```

La fin de semaine publie cinq fois plus que la semaine : les motoneiges sortent le samedi et le dimanche. Quinze secondes après le dernier message, l'abonné s'arrête et fait son bilan :

```
Aucun message depuis 15 s : arrêt.
  telemetrie : doublons : 70
  telemetrie : messages reçus : 3449
  telemetrie : reçus plus d'une heure après la fin : 246
  usine : messages retenus reçus à l'abonnement : 58
```

Les 70 doublons sont ceux de la qualité de service 1, que le rejoueur publie deux fois parce que l'abonné de Boréal les avait reçus deux fois. Les 246 messages en retard viennent de véhicules sortis de la couverture. La dernière ligne s'expliquera à l'étape 6 : ce sont les points retenus de l'étape 2, livrés dès l'abonnement.

Étape 5 — Comparer à l'archive

Théorie : `Theorie_09_DE.md`, section 6.

L'abonné a écrit sept fichiers dans `Projet_Boreal_intermediaire\flux\telemetrie`, un par jour de réception. Ils doivent égaler ceux de `sources\telemetrie` pour la même semaine. Ouvrez la ligne de commande DuckDB depuis `Data_Domain\Projet_Boreal_intermediaire`, et posez deux vues sur le même moule :

```
SET TimeZone = 'America/Toronto';
CREATE OR REPLACE VIEW archive AS
SELECT sujet, CAST(recu_le AS TIMESTAMPTZ) AS recu, qos, CAST(message AS VARCHAR) AS message
FROM read_json('sources/telemetrie/sorties_2026-01-1[0-6].jsonl',
     columns = {sujet: 'VARCHAR', recu_le: 'VARCHAR', qos: 'INTEGER', message: 'JSON'});
CREATE OR REPLACE VIEW abonne AS
SELECT sujet, CAST(recu_le AS TIMESTAMPTZ) AS recu, qos, CAST(message AS VARCHAR) AS message
FROM read_json('flux/telemetrie/*.jsonl',
     columns = {sujet: 'VARCHAR', recu_le: 'VARCHAR', qos: 'INTEGER', message: 'JSON'});
```

Comparez le contenu, dans les deux sens. `EXCEPT ALL` garde les doublons : un message reçu deux fois d'un côté et une seule de l'autre ferait un écart.

© 2026 iAlexMG. Tous droits réservés.

```sql
-- Attendu : 3 449 et 3 449 lignes ; 0 et 0 écart.
SELECT (SELECT count(*) FROM archive) AS archive, (SELECT count(*) FROM abonne) AS abonne,
       (SELECT count(*) FROM (SELECT sujet, qos, message FROM archive
                              EXCEPT ALL SELECT sujet, qos, message FROM abonne)) AS archive_seule,
       (SELECT count(*) FROM (SELECT sujet, qos, message FROM abonne
                              EXCEPT ALL SELECT sujet, qos, message FROM archive)) AS abonne_seul;
```

Seule l'heure de réception diffère, et de peu. Appariez chaque message à son jumeau, par son identifiant et son rang de doublon :

```sql
-- Attendu : 3 449 messages appariés, quelques secondes d'écart en moyenne, moins d'une minute au
-- plus, et aucun qui change de jour.
WITH a AS (SELECT message::JSON->>'message_id' AS id, recu,
                  row_number() OVER (PARTITION BY message::JSON->>'message_id' ORDER BY recu) AS rang
           FROM archive),
     b AS (SELECT message::JSON->>'message_id' AS id, recu,
                  row_number() OVER (PARTITION BY message::JSON->>'message_id' ORDER BY recu) AS rang
           FROM abonne)
SELECT count(*) AS apparies,
       ROUND(AVG(ABS(EPOCH(b.recu) - EPOCH(a.recu))), 1) AS ecart_moyen_s,
       MAX(ABS(EPOCH(b.recu) - EPOCH(a.recu))) AS ecart_max_s,
       count(*) FILTER (WHERE CAST(a.recu AS DATE) <> CAST(b.recu AS DATE)) AS changent_de_jour
FROM a JOIN b USING (id, rang);
```

Sur le poste de l'auteur : 2,7 s d'écart en moyenne, et 54 à 55 s au plus selon le passage. À 3 600 fois, une milliseconde du poste vaut 3,6 secondes de Boréal, et le rejeu ne tombe pas à la milliseconde. Le contenu, lui, est identique : l'archive que le chemin froid charge depuis le Cours 03 est exactement ce qu'un abonné comme le vôtre écrit.

Étape 6 — Le message retenu

Théorie : `Theorie_09_DE.md`, section 5.

Relancez l'abonné, `python -m flux.abonne --silence 10`, puis rejouez de nouveau la journée du 28 octobre depuis l'autre Anaconda Prompt, comme à l'étape 2. L'abonné compte ce qu'il reçoit :

```
Connecté ; session reprise : oui.
Aucun message depuis 10 s : arrêt.
  usine : messages retenus reçus à l'abonnement : 58
  usine : messages reçus : 4048
  usine : points sans valeur : 7
```

La session est reprise : c'est la même que celle de l'étape 4, sous l'identifiant `abonne-01de`. Les 58 retenus sont ceux de l'étape 2, livrés à l'abonnement, avant le rejeu. Les 7 points sans valeur sont une coupure de la zone des motomarines, à 17 h 16, ses sept étiquettes d'un coup, comme au Cours 08.

© 2026 iAlexMG. Tous droits réservés.

Relancez maintenant l'abonné une fois de plus, sans rien rejouer : `python -m flux.abonne`. Les messages retenus arrivent aussitôt, mais ils ne font pas courir le silence, et l'abonné attendrait indéfiniment un message neuf. Au bout de quelques secondes, arrêtez-le par Ctrl+C :

```
Connecté ; session reprise : oui.
Arrêt demandé.
  usine : messages retenus reçus à l'abonnement : 58
```

Un abonné qui arrive après la bataille reçoit d'un coup la dernière valeur de chacune des 58 étiquettes, et rien d'autre : le courtier ne garde que le dernier message de chaque sujet, et seulement s'il est retenu.

Vérifiez enfin que les 4 048 points reçus sont ceux de l'historien. Dans la ligne de commande DuckDB :

```
-- Attendu : 4 048 et 4 048, puis 0 et 0.
CREATE OR REPLACE VIEW historien AS
SELECT TagName AS etiquette, Timestamp AS instant_local, Value AS valeur, Quality AS qualite
FROM read_csv('sources/usine/historien/historien_2025-10-28.csv',
     columns = {'TagName': 'VARCHAR', 'Timestamp': 'TIMESTAMP', 'Value': 'DOUBLE', 'Quality': 'INTEGER'})
WHERE CAST(Timestamp AS DATE) = DATE '2025-10-28';
CREATE OR REPLACE VIEW recus AS
SELECT etiquette, CAST(CAST(instant AS TIMESTAMPTZ) AS TIMESTAMP) AS instant_local, valeur, qualite
FROM read_json('flux/usine/points_2025-10-28.jsonl',
     columns = {etiquette: 'VARCHAR', instant: 'VARCHAR', valeur: 'DOUBLE', qualite: 'INTEGER'});
SELECT (SELECT count(*) FROM historien) AS historien, (SELECT count(*) FROM recus) AS recus,
       (SELECT count(*) FROM (SELECT * FROM historien EXCEPT ALL SELECT * FROM recus)) AS historien_seul,
       (SELECT count(*) FROM (SELECT * FROM recus EXCEPT ALL SELECT * FROM historien)) AS recus_seuls;
```

Le fichier de l'historien porte 4 106 lignes ce jour-là ; les 58 valeurs de départ, que le rejoueur ne publie pas, font la différence.

Étape 7 — Couper l'abonné, et voir ce que la session garde

Théorie : `Theorie_09_DE.md`, section 5.

L'abonné écrit à la suite de ses fichiers : effacez d'abord ceux de la télémétrie, depuis `01_de`, pour repartir d'une comparaison propre.

```
rmdir /s /q ..\flux\telemetrie
python -m flux.abonne
```

Lancez le rejeu de la semaine de télémétrie depuis l'autre Anaconda Prompt, comme à l'étape 4. Au bout d'une quarantaine de secondes, pendant le dimanche 11, arrêtez l'abonné par Ctrl+C. Attendez une demi-minute, pendant que le rejeu continue, puis relancez `python -m flux.abonne`. Il annonce `session reprise : oui`, et reçoit d'un bloc ce que le courtier lui a gardé, avant de reprendre le fil.

Refaites la comparaison de l'étape 5, en reposant les deux vues si vous avez fermé la ligne de commande DuckDB entre-temps : une vue ne vit que le temps de la session. Le contenu est intact, 3 449 lignes de part et d'autre et 0 écart dans les deux sens. L'heure, elle, ne l'est pas : les messages gardés par le courtier portent

© 2026 iAlexMG. Tous droits réservés.

l'heure du retour. Sur trois essais de l'auteur, l'écart le plus grand a atteint de 30 à 33 heures de Boréal, et de 245 à 297 messages ont changé de fichier quotidien. Vos chiffres dépendent du moment exact de la coupure.

Piège — la file d'un absent est bornée. Mosquitto garde à un abonné absent au plus 1 000 messages de qualité 1 par défaut, et jette le reste sans rien dire. Mesuré avec le réglage par défaut, un abonné absent pendant que 2 118 messages passaient n'en a retrouvé que 999. Le `mosquitto.conf` de la plateforme relève la borne à 100 000 : ouvrez-le, la ligne `max_queued_messages` le dit.

Une session persistante continue de remplir sa file tant qu'elle n'est pas fermée : la seconde séance ferait accumuler au courtier, pour un abonné qui n'écoute plus, les 62 726 points d'octobre. Fermez-la, en vous connectant une fois en session propre, puis Ctrl+C au bout de quelques secondes :

```
python -m flux.abonne --session-propre
```

```
Connecté ; session reprise : non.
Arrêt demandé.
  usine : messages retenus reçus à l'abonnement : 58
```

Étape 8 — Versionner

Depuis `01_de` :

```
git add flux/__init__.py flux/commun.py flux/abonne.py config.toml
git commit -m "Cours 09 : l'abonne du courtier MQTT, et l'archive prouvee"
```

C'est la fin de la première séance. Le courtier peut rester démarré ; `docker compose stop mqtt` l'arrête sans rien perdre, sinon ses messages retenus, qu'il ne garde qu'en mémoire.

Avant la seconde séance — Kafka, et le mot de passe de la base

Dans Ubuntu, depuis `plateforme`, démarrez Kafka et la base de séries temporelles du Cours 08, et le courtier s'il est arrêté :

```
docker compose up -d mqtt kafka series
docker compose logs kafka | grep "Kafka Server started"
```

```
boreal_kafka  | [2026-09-23 06:55:56,195] INFO [KafkaRaftServer nodeId=1] Kafka Server started (kafka.server.KafkaRaftServer)
```

L'image pèse 686 Mo et ne se tire qu'une fois ; Kafka répond ensuite en moins de cinq secondes. Si la commande ne rend rien, relancez-la quelques secondes plus tard.

Le code de ce cours écrira dans la base de séries temporelles depuis Windows, et il lui faut le mot de passe choisi au Cours 08, celui du `.env` de la plateforme. Ouvrez le `.env` de votre dépôt, `notepad .env` depuis `01_de`, et ajoutez :

© 2026 iAlexMG. Tous droits réservés.

```
SERIES_MOT_DE_PASSE=le-meme-que-dans-plateforme-env
```

Étape 9 — Créer les sujets, et choisir leurs partitions

Théorie : `Theorie_09_DE.md`, sections 7 et 8.

La plateforme interdit à Kafka de créer un sujet de lui-même (`KAFKA_AUTO_CREATE_TOPICS_ENABLE` à `false`) : une faute de frappe doit être une erreur, pas un sujet neuf. Les sujets se créent donc à la main, avec leurs partitions. L'outil est un script de Kafka, dans son conteneur ; une variable d'Ubuntu évite de réécrire son chemin à chaque fois. Depuis `plateforme` :

```
K="docker compose exec kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092"
$K --create --topic usine.points --partitions 3 --replication-factor 1
$K --create --topic vehicules.sorties --partitions 3 --replication-factor 1
$K --create --topic usine.alertes --partitions 1 --replication-factor 1
$K --list
```

Chaque création répond deux lignes :

```
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is
best to use either, but not both.
Created topic usine.points.
```

L'avertissement dit qu'un point et un souligné deviennent le même caractère dans les noms des mesures de Kafka : `usine.points` et `usine_points` s'y confondraient. Boréal n'emploie que des points, et l'avertissement est sans conséquence. `$K --list` rend les trois sujets.

Regardez comment Kafka range le premier :

```
$K --describe --topic usine.points
```

```
Topic: usine.points TopicId: 1-TU7k4wSNGtXdCHysprow PartitionCount: 3   ReplicationFactor: 1    Configs: min.insync.replicas=1
    Topic: usine.points Partition: 0    Leader: 1   Replicas: 1 Isr: 1  Elr:    LastKnownElr:
    Topic: usine.points Partition: 1    Leader: 1   Replicas: 1 Isr: 1  Elr:    LastKnownElr:
    Topic: usine.points Partition: 2    Leader: 1   Replicas: 1 Isr: 1  Elr:    LastKnownElr:
```

Trois partitions, chacune tenue par le nœud 1, sans aucune copie. Votre `TopicId` sera autre. Trois partitions pour les points et les sorties : trois consommateurs d'un même groupe pourront s'en partager la lecture. Une seule pour les alertes : elles sont rares, et on les veut dans l'ordre.

Étape 10 — Écrire le pont

Théorie : `Theorie_09_DE.md`, sections 8 et 10.

Le pont s'abonne au courtier comme l'abonné, et écrit chaque message dans un sujet Kafka. Créez-le, `notepad flux\pont.py` depuis `01_de`. Son début choisit la clé :

© 2026 iAlexMG. Tous droits réservés.

```python
# -*- coding: utf-8 -*-
"""pont.py : relaie le courtier MQTT vers Kafka, et n'acquitte qu'une fois Kafka a écrit.

Usage, depuis 01_de : python -m flux.pont
"""
import argparse
import json

from confluent_kafka import Producer

from flux.commun import Recepteur, ecouter, iso, reglages

SUJET_USINE = "usine.points"
SUJET_SORTIES = "vehicules.sorties"


def actif(etiquette: str) -> str:
    """L'équipement d'une étiquette : UTL.CMP2.VIBRATION devient UTL-CMP2."""
    zone, equipement = etiquette.split(".")[:2]
    return f"{zone}-{equipement}"


class Pont(Recepteur):
    manuel = True                           # l'accusé attend la confirmation de Kafka

    def __init__(self, producteur):
        super().__init__()
        self.producteur = producteur
        self.erreurs = 0
```

Un point porte pour clé son équipement : les quatre étiquettes du compresseur 2 tomberont dans la même partition, et l'alerte les lira dans l'ordre. Le pont se déclare `manuel` : paho ne l'acquittera pas de lui-même. Le cœur du pont :

```python
    def traiter(self, flux, msg, recu):
        charge = json.loads(msg.payload)
        if flux == "usine":
            sujet, cle = SUJET_USINE, actif(charge["etiquette"])
            valeur = dict(charge, recu_le=iso(recu))
        else:
            sujet, cle = SUJET_SORTIES, charge["sernr"]
            valeur = {"sujet": msg.topic, "recu_le": iso(recu), "qos": msg.qos, "message": charge}

        def confirmer(erreur, _message, mid=msg.mid, qos=msg.qos):
```


© 2026 iAlexMG. Tous droits réservés.

```
            if erreur is None:
                self.client.ack(mid, qos)   # Kafka a écrit : le courtier peut oublier le message
                self.compte[f"{flux} : relayés vers {sujet}"] += 1
            else:
                self.erreurs += 1           # pas d'accusé : le courtier le relivrera
                print(f"Kafka refuse un message de {sujet} : {erreur}", flush=True)

        self.producteur.produce(sujet, key=cle, on_delivery=confirmer,
                                value=json.dumps(valeur, ensure_ascii=False, separators=(",", ":")))
        self.producteur.poll(0)             # sert les rappels déjà prêts, sans attendre

    def attendre(self, secondes):
        self.producteur.poll(secondes)      # les rappels de livraison arrivent ici

    def terminer(self):
        reste = self.producteur.flush(10)
        if reste:
            print(f"{reste} messages sans confirmation de Kafka : non acquittés, ils reviendront.")
```

L'accusé part de `confirmer()`, le rappel que le producteur appelle quand Kafka a écrit le message, ou quand il y renonce. Tant que Kafka n'a pas confirmé, le courtier garde le message, et le relivrera si le pont tombe. `attendre()` remplace le sommeil de la boucle d'écoute par `producteur.poll()`, qui sert ces rappels. La porte d'entrée :

```
def principal():
    p = argparse.ArgumentParser(description="Relaie le courtier MQTT de Boréal vers Kafka.")
    p.add_argument("--silence", type=float, default=15,
                   help="s'arrêter après tant de secondes sans message")
    a = p.parse_args()

    producteur = Producer({
        "bootstrap.servers": reglages()["kafka"],
        "client.id": "pont-01de",
        "acks": "all",                  # la confirmation vient une fois le message écrit par le nœud
        "enable.idempotence": True,     # une nouvelle tentative du producteur n'écrit pas deux fois
    })
    pont = Pont(producteur)
    ecouter(pont, "pont-01de", persistante=True, silence=a.silence)
    for cle in sorted(pont.compte):
        print(f"  {cle} : {pont.compte[cle]}")
    if pont.erreurs:
        print(f"  refusés par Kafka, non acquittés : {pont.erreurs}")
```


© 2026 iAlexMG. Tous droits réservés.

```
if __name__ == "__main__":
    principal()
```

`acks` à `all` demande la confirmation une fois le message écrit ; `enable.idempotence` fait qu'une nouvelle tentative du producteur, après un accusé perdu en route, n'écrive pas le message deux fois.

Piège — attendre chaque confirmation une à une. La façon la plus simple d'acquitter après Kafka est d'appeler `producteur.flush()` à la fin de `traiter()`, et de laisser paho acquitter au retour. Elle est correcte, et inutilisable sur le poste de l'auteur : chaque confirmation y coûte 44 ms dès que le message dépasse 200 octets environ, contre 0,5 ms en dessous, parce que le relais qui mène de Windows aux ports de WSL 2 retient ces envois. Le même client lancé depuis Linux répond en 0,4 ms à toutes les tailles. Un pont écrit ainsi plafonnait à 22 messages par seconde, et datait la télémétrie d'une fin de semaine jusqu'à deux jours trop tard. Le rappel laisse le producteur grouper les messages en attente.

Étape 11 — Écrire le consommateur, et les tables qui dédoublonnent

Théorie : `Theorie_09_DE.md`, sections 9 et 13.

La base de séries temporelles reçoit un schéma neuf, `temps_reel`, à côté de ce que le Cours 08 y a chargé. Créez son script, `notepad sql\temps_reel.sql` depuis `01_de` :

```
-- temps_reel.sql : les tables du chemin chaud dans TimescaleDB (Cours 09). La cle primaire
-- de chaque table fait le dedoublonnage : ON CONFLICT DO NOTHING ecarte ce qui existe deja.

CREATE SCHEMA IF NOT EXISTS temps_reel;

-- Les points de l'usine, un par etiquette et par instant. L'instant est absolu, resolu par le
-- rejoueur ; recu_le est l'heure de Boreal a laquelle le pont l'a recu.
CREATE TABLE IF NOT EXISTS temps_reel.points (
  instant   TIMESTAMPTZ      NOT NULL,
  etiquette TEXT             NOT NULL,
  valeur    DOUBLE PRECISION,
  qualite   SMALLINT         NOT NULL,
  recu_le   TIMESTAMPTZ      NOT NULL,
  PRIMARY KEY (etiquette, instant)
);
SELECT create_hypertable('temps_reel.points', by_range('instant', INTERVAL '7 days'),
                         if_not_exists => TRUE);

-- Les resumes de sortie, un par message : ses doublons n'entrent pas. La fin de la sortie et
-- l'heure de reception se gardent cote a cote, parce que leur ecart est le retard.
CREATE TABLE IF NOT EXISTS temps_reel.sorties (
  message_id TEXT        PRIMARY KEY,
```


© 2026 iAlexMG. Tous droits réservés.

```
  sernr      TEXT        NOT NULL,
  fin        TIMESTAMPTZ NOT NULL,
  recu_le    TIMESTAMPTZ NOT NULL,
  codes      TEXT[]      NOT NULL,
  message    JSONB       NOT NULL
);

-- Les alertes que leve flux/alerte.py, une ligne par changement d'etat.
CREATE TABLE IF NOT EXISTS temps_reel.alertes (
  actif         TEXT             NOT NULL,
  niveau        TEXT             NOT NULL,
  etat          TEXT             NOT NULL,
  fenetre_debut TIMESTAMPTZ      NOT NULL,
  fenetre_fin   TIMESTAMPTZ      NOT NULL,
  moyenne       DOUBLE PRECISION NOT NULL,
  recu_le       TIMESTAMPTZ      NOT NULL,
  PRIMARY KEY (actif, niveau, fenetre_debut)
);

-- Attendu au premier lancement : trois tables dans temps_reel, dont une hypertable.
SELECT table_name FROM information_schema.tables WHERE table_schema = 'temps_reel' ORDER BY 1;
SELECT hypertable_name FROM timescaledb_information.hypertables
WHERE hypertable_schema = 'temps_reel';
```

Chaque table a une clé primaire qui identifie le fait. C'est elle qui écartera les doublons, sans une ligne de code qui les cherche. Les contraintes et l'insertion qui cède devant un conflit (`ON CONFLICT`) sont au Cours 03 de la formation SQL avancé, sections 3 et 6. Passez le script à la base, depuis Ubuntu, dans `plateforme`. Le fichier est lu par le terminal d'Ubuntu et donné à `psql` par son entrée, d'où le `-T`, qui n'attache pas de terminal au conteneur :

```
docker compose exec -T series psql -U postgres -d usine < ../01_de/sql/temps_reel.sql
```

La sortie finit sur les trois tables du schéma, puis sur la seule qui soit une hypertable :

```
 table_name
------------
 alertes
 points
 sorties
(3 rows)

 hypertable_name
-----------------
 points
(1 row)
```


© 2026 iAlexMG. Tous droits réservés.

Le consommateur, maintenant : `notepad flux\vers_series.py`. D'abord, pour chaque sujet, la table visée et la ligne qu'on tire d'un message :

```
# -*- coding: utf-8 -*-
"""vers_series.py : lit Kafka par lots et les écrit dans TimescaleDB, la base d'abord, le
décalage ensuite.

Usage, depuis 01_de : python -m flux.vers_series
"""
import argparse
import json
import time
from collections import Counter

import psycopg
from confluent_kafka import Consumer, KafkaException
from psycopg.types.json import Jsonb

from flux.commun import reglages
from pipeline.config import secret


def point(v):
    return v["instant"], v["etiquette"], v["valeur"], v["qualite"], v["recu_le"]


def sortie(v):
    m = v["message"]
    return m["message_id"], m["sernr"], m["fin"], v["recu_le"], m["codes"], Jsonb(m)


def alerte(v):
    return (v["actif"], v["niveau"], v["etat"], v["fenetre_debut"], v["fenetre_fin"],
            v["moyenne"], v["recu_le"])


# Pour chaque sujet : la table visée, ses colonnes, et la ligne qu'on tire d'un message.
TABLES = {
    "usine.points": ("temps_reel.points",
                     ["instant", "etiquette", "valeur", "qualite", "recu_le"], point),
    "vehicules.sorties": ("temps_reel.sorties",
                          ["message_id", "sernr", "fin", "recu_le", "codes", "message"], sortie),
    "usine.alertes": ("temps_reel.alertes",
```


© 2026 iAlexMG. Tous droits réservés.

```
                      ["actif", "niveau", "etat", "fenetre_debut", "fenetre_fin", "moyenne",
                       "recu_le"], alerte),
}
```

Puis l'écriture d'un lot, et l'ouverture du consommateur et de la base :

```
def ecrire(cur, sujet, lignes):
    """Un lot dans sa table, par une table temporaire ; rend le nombre de lignes insérées."""
    table, colonnes, _ = TABLES[sujet]
    lot = "lot_" + table.split(".")[1]
    cur.execute(f"CREATE TEMP TABLE IF NOT EXISTS {lot} (LIKE {table}) ON COMMIT DELETE ROWS")
    liste = ", ".join(colonnes)
    with cur.copy(f"COPY {lot} ({liste}) FROM STDIN") as copie:
        for ligne in lignes:
            copie.write_row(ligne)
    cur.execute(f"INSERT INTO {table} ({liste}) SELECT {liste} FROM {lot} "
                "ON CONFLICT DO NOTHING")
    return cur.rowcount


def ouvrir(r):
    """Le consommateur du groupe vers-series, et la connexion à la base."""
    consommateur = Consumer({
        "bootstrap.servers": r["kafka"],
        "group.id": "vers-series",
        "auto.offset.reset": "earliest",   # un groupe neuf commence au plus ancien message gardé
        "enable.auto.commit": False,       # la position ne s'enregistre qu'après la base
    })
    consommateur.subscribe(list(TABLES))
    base = psycopg.connect(host=r["series_serveur"], port=r["series_port"],
                           dbname=r["series_base"], user=r["series_utilisateur"],
                           password=secret("SERIES_MOT_DE_PASSE"),
                           autocommit=True)    # chaque lot ouvre et valide sa transaction
    return consommateur, base
```

Un lot entre par `COPY` dans une table temporaire, puis dans sa table par `INSERT … ON CONFLICT DO NOTHING` : `rowcount` rend alors le nombre de lignes vraiment insérées. Le groupe `vers-series` n'enregistre pas son décalage de lui-même (`enable.auto.commit` à faux). La boucle :

```
def principal():
    p = argparse.ArgumentParser(description="Écrit les sujets Kafka de Boréal dans TimescaleDB.")
    p.add_argument("--silence", type=float, default=15,
                   help="s'arrêter après tant de secondes sans message")
    a = p.parse_args()
```


© 2026 iAlexMG. Tous droits réservés.

```
    consommateur, base = ouvrir(reglages())
    lus, inseres = Counter(), Counter()
    dernier = None                        # le silence ne compte qu'après le premier message
    print("Lecture de", ", ".join(TABLES), ": Ctrl+C pour arrêter.", flush=True)
    try:
        while dernier is None or time.time() - dernier < a.silence:
            messages = consommateur.consume(num_messages=500, timeout=1.0)
            if not messages:
                continue
            dernier = time.time()
            lots = {}
            for m in messages:
                if m.error():
                    raise KafkaException(m.error())
                ligne = TABLES[m.topic()][2](json.loads(m.value()))
                lots.setdefault(m.topic(), []).append(ligne)
            with base.transaction(), base.cursor() as cur:
                for sujet, lignes in lots.items():
                    lus[sujet] += len(lignes)
                    inseres[sujet] += ecrire(cur, sujet, lignes)
            consommateur.commit(asynchronous=False)     # après la base, jamais avant
        print(f"Aucun message depuis {a.silence:g} s : arrêt.")
    except KeyboardInterrupt:
        print("Arrêt demandé.")
    consommateur.close()
    base.close()
    for sujet in TABLES:
        if lus[sujet]:
            print(f"  {sujet} : {lus[sujet]} lus, {inseres[sujet]} insérés, "
                  f"{lus[sujet] - inseres[sujet]} déjà présents")


if __name__ == "__main__":
    principal()
```

L’ordre des deux validations est la règle du fichier : la base d’abord, par la transaction du lot, puis le décalage dans Kafka. Une panne entre les deux fait relire le lot, que la clé primaire écarte.

Étape 12 — Écrire les fenêtres et l’alerte, et les tester sans Kafka

Théorie : `Theorie_09_DE.md`, sections 11 et 12.

© 2026 iAlexMG. Tous droits réservés.

La logique de l'alerte vit dans un module sans aucune dépendance, pour se tester sans rien démarrer. Créez `notepad flux\fenetres.py` :

```
# -*- coding: utf-8 -*-
"""fenetres.py : les fenêtres de 15 minutes et l'alarme à bande morte, sans aucune dépendance."""
from datetime import datetime, timedelta

FENETRE = timedelta(minutes=15)


def debut_fenetre(t: datetime) -> datetime:
    """Le début de la fenêtre de 15 minutes qui contient t."""
    return t.replace(minute=t.minute - t.minute % 15, second=0, microsecond=0)


class Fenetres:
    """Les fenêtres d'un équipement : vibration pondérée par le temps, pendant la marche seulement.

    Le temps qui compte est celui des points. Une fenêtre se ferme quand arrive un point plus récent
    que sa fin, jamais selon l'horloge du poste. Et une moyenne pondérée a besoin de la valeur en
    vigueur quand la fenêtre s'ouvre : l'objet garde le dernier état de marche, la dernière
    vibration et l'instant du dernier point.
    """

    def __init__(self):
        self.marche = None                  # le dernier état de marche de qualité bonne
        self.vibration = None               # la dernière vibration de qualité bonne
        self.depuis = None                  # l'instant du dernier point vu
        self.ouvertes = {}                  # début -> [somme valeur x secondes, secondes]
```

`Fenetres` garde l'état d'un équipement. Sa méthode `point()` en est le cœur :

```
    def point(self, etiquette, instant, valeur, qualite):
        """Ajoute un point. Rend les fenêtres qu'il ferme, (début, moyenne, secondes de marche), ou
        None si le point est antérieur au dernier vu : arrivé en retard, il ne compte pas."""
        if self.depuis is not None and instant < self.depuis:
            return None
        if self.depuis is not None and self.marche == 1 and self.vibration is not None:
            a = self.depuis                 # la valeur en vigueur a tenu de depuis à instant
            while a < instant:
                d = debut_fenetre(a)
                b = min(instant, d + FENETRE)
                cumul = self.ouvertes.setdefault(d, [0.0, 0.0])
                cumul[0] += self.vibration * (b - a).total_seconds()
```


© 2026 iAlexMG. Tous droits réservés.

```
            cumul[1] += (b - a).total_seconds()
            a = b
        fermees = []
        for d in sorted(k for k in self.ouvertes if k + FENETRE <= instant):
            somme, secondes = self.ouvertes.pop(d)
            fermees.append((d, somme / secondes, secondes))
        self.depuis = instant
        bonne = valeur if qualite == 192 else None
        if etiquette.endswith(".MARCHE"):
            self.marche = bonne
        elif etiquette.endswith(".VIBRATION"):
            self.vibration = bonne
        return fermees
```

La valeur en vigueur a tenu depuis le point précédent jusqu'à celui-ci : la boucle découpe cette durée aux frontières des quarts d'heure, et n'en compte que ce qui tombe pendant la marche. Puis chaque fenêtre dont la fin est dépassée se ferme. C'est l'instant du point qui ferme, jamais l'horloge du poste. L'alarme, ensuite, dans le même fichier :

```
class Alarme:
    """Un niveau d'alarme : levé au seuil, retombé seulement sous le seuil moins la bande morte.

    Sans bande morte, une moyenne qui oscille autour du seuil lève et fait retomber l'alarme à
    chaque fenêtre, et l'opérateur cesse de la lire.
    """

    def __init__(self, niveau, seuil, bande_morte=0.5):
        self.niveau, self.seuil, self.bande_morte = niveau, seuil, bande_morte
        self.levee = False

    def juger(self, moyenne):
        """« levée », « retombée », ou None quand l'état ne change pas."""
        if not self.levee and moyenne >= self.seuil:
            self.levee = True
            return "levée"
        if self.levee and moyenne < self.seuil - self.bande_morte:
            self.levee = False
            return "retombée"
        return None
```

Le programme de l'alerte branche ces deux classes sur Kafka : `notepad flux\alerte.py`. Il juge d'abord les fenêtres fermées :

```
# -*- coding: utf-8 -*-
"""alerte.py : les fenêtres de 15 minutes sur la vibration des compresseurs, et leur alerte.
```


© 2026 iAlexMG. Tous droits réservés.

```
    Usage, depuis 01_de : python -m flux.alerte [--groupe G] [--seuil 4.5] [--essai]
    """
    import argparse
    import json
    import time
    from datetime import datetime, timezone

    from confluent_kafka import Consumer, KafkaException, Producer

    from flux.commun import iso, reglages
    from flux.fenetres import FENETRE, Alarme, Fenetres

    MARCHE_MINIMALE = 600                   # secondes de marche pour qu'une fenêtre soit jugée
    ARRET = 7.1                             # mm/s : l'arrêt immédiat de la procédure


    def juger(actif, fermees, alarmes, seuil):
        """Les changements d'état que provoquent les fenêtres qu'un point vient de fermer."""
        alertes = []
        for debut, moyenne, secondes in fermees:
            if secondes < MARCHE_MINIMALE:
                continue                    # le compresseur a trop peu tourné pour qu'on juge
            for niveau, limite in (("ALERTE", seuil), ("ARRET", ARRET)):
                alarme = alarmes.setdefault((actif, niveau), Alarme(niveau, limite))
                etat = alarme.juger(moyenne)
                if etat is not None:
                    alertes.append({"actif": actif, "niveau": niveau, "etat": etat,
                                    "fenetre_debut": iso(debut), "fenetre_fin": iso(debut + FENETRE),
                                    "moyenne": round(moyenne, 3)})
        return alertes
```

Puis il annonce, et ouvre son consommateur et son producteur :

```
    def annoncer(alerte):
        """Une ligne par changement d'état, datée de la fin de la fenêtre qui l'a provoqué."""
        print(f"{alerte['fenetre_fin'][:16].replace('T', ' ')}  {alerte['actif']}  "
              f"{alerte['niveau']} {alerte['etat']} : {alerte['moyenne']:.3f} mm/s "
              f"sur la fenêtre de {alerte['fenetre_debut'][11:16]}", flush=True)


    def ouvrir(groupe):
        """Le consommateur du groupe demandé, et le producteur des alertes."""
```


© 2026 iAlexMG. Tous droits réservés.

```
    r = reglages()
    consommateur = Consumer({"bootstrap.servers": r["kafka"], "group.id": groupe,
                             "auto.offset.reset": "earliest"})
    consommateur.subscribe(["usine.points"])
    producteur = Producer({"bootstrap.servers": r["kafka"], "acks": "all",
                           "enable.idempotence": True})
    return consommateur, producteur
```

Enfin, la boucle, qui ne garde que les compresseurs d'air :

```
def principal():
    p = argparse.ArgumentParser(description="Lève l'alerte de vibration des compresseurs.")
    p.add_argument("--groupe", default="alerte-vibration", help="le groupe de consommateurs")
    p.add_argument("--seuil", type=float, default=4.5, help="le seuil d'alerte, en mm/s")
    p.add_argument("--silence", type=float, default=15,
                   help="s'arrêter après tant de secondes sans message")
    p.add_argument("--essai", action="store_true", help="afficher sans écrire dans Kafka")
    a = p.parse_args()

    consommateur, producteur = ouvrir(a.groupe)
    fenetres, alarmes, en_retard = {}, {}, 0
    dernier = None                       # le silence ne compte qu'après le premier message
    print(f"Groupe {a.groupe}, alerte à {a.seuil:g} mm/s, arrêt à {ARRET:g} mm/s.", flush=True)
    try:
        while dernier is None or time.time() - dernier < a.silence:
            m = consommateur.poll(1.0)
            if m is None:
                continue
            if m.error():
                raise KafkaException(m.error())
            dernier = time.time()
            actif = m.key().decode()
            if not actif.startswith("UTL-CMP"):
                continue
            v = json.loads(m.value())
            instant = datetime.fromisoformat(v["instant"]).astimezone(timezone.utc)
            f = fenetres.setdefault(actif, Fenetres())
            fermees = f.point(v["etiquette"], instant, v["valeur"], v["qualite"])
            if fermees is None:
                en_retard += 1
                continue
            for alerte in juger(actif, fermees, alarmes, a.seuil):
                alerte["recu_le"] = v["recu_le"]
```


© 2026 iAlexMG. Tous droits réservés.

```
                annoncer(alerte)
                if not a.essai:
                    producteur.produce("usine.alertes", key=actif,
                                       value=json.dumps(alerte, ensure_ascii=False))
            producteur.poll(0)
        print(f"Aucun message depuis {a.silence:g} s : arrêt.")
    except KeyboardInterrupt:
        print("Arrêt demandé.")
    producteur.flush(10)
    consommateur.close()
    print(f"  points écartés parce qu'arrivés en retard : {en_retard}")


if __name__ == "__main__":
    principal()
```

Les tests d'abord, avant tout flux. Créez `notepad tests\test_flux.py` :

```
# -*- coding: utf-8 -*-
"""test_flux.py : les fenêtres et l'alarme, vérifiées sans courtier ni Kafka."""
from datetime import datetime, timedelta, timezone

from flux.fenetres import Alarme, Fenetres

T0 = datetime(2025, 10, 22, 12, 0, tzinfo=timezone.utc)


def minutes(n):
    return T0 + timedelta(minutes=n)


def test_moyenne_ponderee_pendant_la_marche():
    f = Fenetres()
    f.point("UTL.CMP2.MARCHE", minutes(0), 1, 192)
    f.point("UTL.CMP2.VIBRATION", minutes(0), 2.0, 192)
    f.point("UTL.CMP2.VIBRATION", minutes(5), 4.0, 192)       # 2,0 pendant 5 min, puis 4,0
    f.point("UTL.CMP2.MARCHE", minutes(12), 0, 192)            # arrêt : 7 min à 4,0, pas plus
    fermees = f.point("UTL.CMP2.VIBRATION", minutes(16), 0.1, 192)
    assert len(fermees) == 1
    debut, moyenne, secondes = fermees[0]
    assert debut == T0
    assert secondes == 12 * 60
    assert abs(moyenne - (2.0 * 5 + 4.0 * 7) / 12) < 1e-9     # 3,1667, et non 3,0 en moyenne simple
```


© 2026 iAlexMG. Tous droits réservés.

```python
def test_point_en_retard_ecarte():
    f = Fenetres()
    f.point("UTL.CMP2.MARCHE", minutes(0), 1, 192)
    f.point("UTL.CMP2.VIBRATION", minutes(0), 2.0, 192)
    f.point("UTL.CMP2.VIBRATION", minutes(10), 2.0, 192)
    assert f.point("UTL.CMP2.VIBRATION", minutes(4), 9.9, 192) is None
    fermees = f.point("UTL.CMP2.VIBRATION", minutes(15), 2.0, 192)
    assert [round(m, 6) for _, m, _ in fermees] == [2.0]


def test_alarme_a_bande_morte():
    alarme = Alarme("ALERTE", 4.5)
    etats = [alarme.juger(m) for m in (4.4, 4.6, 4.3, 4.55, 4.1, 3.9, 4.2, 4.5)]
    assert etats == [None, "levée", None, None, None, "retombée", None, "levée"]
```

Le premier test calcule à la main ce que la fenêtre doit rendre : 2,0 pendant cinq minutes, 4,0 pendant sept, puis l'arrêt, soit 3,1667 et non 3,0, qu'une moyenne simple des deux valeurs aurait donné. Lancez tous les tests du dépôt, depuis `01_de` :

```
pytest
```

Après l'en-tête de la session, `pytest` donne une ligne par fichier de tests :

```
tests\test_flux.py ...                                                   [ 37%]
tests\test_pipeline.py .....                                             [100%]

============================== 8 passed in 0.49s ==============================
```

Les trois tests du chemin chaud et les cinq du pipeline, en moins d'une seconde, sans un conteneur.

Étape 13 — Lancer toute la chaîne sur octobre

Théorie : `Theorie_09_DE.md`, sections 10 et 12.

Quatre programmes tournent ensemble. Ouvrez quatre Anaconda Prompt. Dans les trois premiers, depuis `01_de`, lancez chacun un programme, dans cet ordre :

```
python -m flux.pont
python -m flux.vers_series
python -m flux.alerte
```

Chacun attend son premier message. Dans le quatrième, depuis `Data_Domain`, rejouez l'usine du 13 au 31 octobre 2025 : deux jours avant la dérive, et trois jours après la réparation.

© 2026 iAlexMG. Tous droits réservés.

```
python _univers/rejoueurs/rejouer.py usine --projet Projet_Boreal_intermediaire --depuis 2025-10-13 --jusqua 2025-10-31
```

Le rejeu dure sept minutes et demie. Regardez le terminal de l'alerte : il reste muet huit jours de Boréal, puis écrit ses lignes au moment où Boréal les aurait vues.

```
Groupe alerte-vibration, alerte à 4.5 mm/s, arrêt à 7.1 mm/s.
2025-10-22 08:15  UTL-CMP2  ALERTE levée : 4.545 mm/s sur la fenêtre de 08:00
2025-10-27 06:15  UTL-CMP2  ARRET levée : 7.554 mm/s sur la fenêtre de 06:00
2025-10-30 14:00  UTL-CMP2  ALERTE retombée : 2.503 mm/s sur la fenêtre de 13:45
2025-10-30 14:00  UTL-CMP2  ARRET retombée : 2.503 mm/s sur la fenêtre de 13:45
```

Quinze secondes après la fin du rejeu, chaque programme fait son bilan. Le rejoueur : `62726 messages publiés en 456 s.` Le pont :

```
usine : messages retenus reçus à l'abonnement : 58
usine : relayés vers usine.points : 62726
```

La première ligne n'apparaît que si le courtier a gardé ses messages retenus depuis la première séance : ils sont acquittés et écartés, jamais relayés. Le consommateur :

```
usine.points : 62726 lus, 62726 insérés, 0 déjà présents
usine.alertes : 4 lus, 4 insérés, 0 déjà présents
```

Et l'alerte : `points écartés parce qu'arrivés en retard : 0`. Rien de perdu entre les maillons, rien de doublé, et les quatre changements d'état de l'alerte sont dans la base.

Étape 14 — Lire l'alerte dans la base, et son avance

Théorie : `Theorie_09_DE.md`, sections 1 et 12.

Ouvrez `psql` sur la base, depuis Ubuntu, dans `plateforme`, et faites-lui parler l'heure de l'Est :

```
docker compose exec series psql -U postgres -d usine
```

```
SET TIME ZONE 'America/Toronto';
-- Attendu : 62 726 points, 58 étiquettes, du 13 octobre à minuit au 31 octobre à 23 h 58.
SELECT count(*) AS points, count(DISTINCT etiquette) AS etiquettes, min(instant), max(instant)
FROM temps_reel.points;
```

Puis l'avance de chaque niveau sur le déclenchement, que la GMAO date du 28 octobre à 10 h 44 :

```
SELECT niveau, fenetre_fin, TIMESTAMPTZ '2025-10-28 10:44:00-04' - fenetre_fin AS avance
FROM temps_reel.alertes WHERE etat = 'levée' ORDER BY fenetre_fin;
```


© 2026 iAlexMG. Tous droits réservés.

```
 niveau |      fenetre_fin       |     avance
--------+------------------------+-----------------
 ALERTE | 2025-10-22 08:15:00-04 | 6 days 02:29:00
 ARRET  | 2025-10-27 06:15:00-04 | 1 day 04:29:00
(2 rows)
```

Six jours, deux heures et vingt-neuf minutes. Mettez-les en regard du bon de travail que le Labo 08 a lu dans la GMAO : `BT-2025-00739`, ouvert le 22 octobre à 9 h 12 pour un bruit anormal, et fermé à 10 h 04 sur « rien constaté ». L'alerte était levée depuis 57 minutes quand le technicien est arrivé.

Étape 15 — Relire le flux par un second groupe

Théorie : `Theorie_09_DE.md`, sections 7 et 9.

Kafka a gardé les 62 726 points. Regardez où en est le groupe du consommateur, depuis Ubuntu :

```
docker compose exec kafka /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 --describe --group vers-series
```

```
GROUP           TOPIC             PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID     HOST            CLIENT-ID
vers-series     usine.alertes     0          4               4               0               -               -               -
vers-series     usine.points      0          24812           24812           0               -               -               -
vers-series     usine.points      1          21765           21765           0               -               -               -
vers-series     usine.points      2          16149           16149           0               -               -               -
```

Le retard du groupe (`LAG`) est nul partout : il a tout lu. Les trois partitions d'`usine.points` ne sont pas égales, 24 812, 21 765 et 16 149 messages, parce que dix-sept équipements ne se répartissent pas également entre trois partitions.

Posez maintenant une autre question au même flux, sans rien rejouer : à 4,0 mm/s au lieu de 4,5, quand l'alerte se serait-elle levée. Un groupe neuf commence au plus ancien message gardé, et `--essai` l'empêche d'écrire ses alertes dans `usine.alertes`, où elles se mêleraient à celles de la procédure. Depuis `01_de` :

```
python -m flux.alerte --groupe relecture --seuil 4.0 --essai --silence 5
```

```
Groupe relecture, alerte à 4 mm/s, arrêt à 7.1 mm/s.
2025-10-21 06:45  UTL-CMP2  ALERTE levée : 4.028 mm/s sur la fenêtre de 06:30
2025-10-27 06:15  UTL-CMP2  ARRET levée : 7.554 mm/s sur la fenêtre de 06:00
2025-10-30 14:00  UTL-CMP2  ALERTE retombée : 2.503 mm/s sur la fenêtre de 13:45
2025-10-30 14:00  UTL-CMP2  ARRET retombée : 2.503 mm/s sur la fenêtre de 13:45
```

Un jour d'avance de plus, pour un seuil plus bas. Les lignes s'affichent en moins d'une seconde : le groupe relit tout le sujet à la vitesse du disque, sans attendre le rythme de l'usine, et le courtier MQTT n'en sait rien.

© 2026 iAlexMG. Tous droits réservés.

Étape 16 — Faire passer la télémétrie, et compter ses retards

Théorie : `Theorie_09_DE.md`, sections 11, 13 et 14.

Relancez le pont et le consommateur dans deux Anaconda Prompt, depuis `01_de`, puis rejouez la semaine de télémétrie depuis `Data_Domain`, comme à l'étape 4. Le pont relaie les 3 449 messages ; le consommateur fait son bilan :

```
vehicules.sorties : 3449 lus, 3379 insérés, 70 déjà présents
```

Les 70 doublons de la qualité de service 1 ont traversé le courtier, le pont et Kafka, et se sont arrêtés sur la clé primaire de `temps_reel.sorties`. Dans `psql`, mesurez maintenant les retards, en fenêtres de 15 minutes sur la fin des sorties :

```
SET TIME ZONE 'America/Toronto';
SELECT count(*) AS sorties,
       count(*) FILTER (WHERE recu_le - fin > INTERVAL '1 hour') AS plus_d_une_heure,
       count(*) FILTER (WHERE recu_le > time_bucket(INTERVAL '15 minutes', fin)
                                      + INTERVAL '15 minutes') AS apres_la_fenetre,
       count(*) FILTER (WHERE recu_le > time_bucket(INTERVAL '15 minutes', fin)
                                      + INTERVAL '15 minutes' + INTERVAL '1 hour') AS apres_le_filigrane,
       count(*) FILTER (WHERE fin < TIMESTAMPTZ '2026-01-10 00:00:00-05') AS finies_avant,
       max(recu_le - fin) AS retard_max
FROM temps_reel.sorties;
```

```
 sorties | plus_d_une_heure | apres_la_fenetre | apres_le_filigrane | finies_avant |   retard_max
---------+------------------+------------------+--------------------+--------------+-----------------
    3379 |              246 |              369 |                242 |           18 | 4 days 00:00:26
(1 row)
```

Un moteur qui fermerait chaque fenêtre une heure après sa fin écarterait environ 240 sorties, et elles ne sont pas perdues pour autant : le chemin froid les ramasse dans l'archive du lendemain. C'est le partage de Lambda. Vos chiffres des colonnes `apres_la_fenetre` et `apres_le_filigrane` peuvent différer de quelques unités, et c'est normal : l'auteur a obtenu de 369 à 374 et 241 ou 242 sur trois rejeux, contre 306 et 241 dans l'archive elle-même. Le rejeu date chaque réception de quelques secondes plus tard, et ce qui arrivait juste avant la fin d'une fenêtre passe de l'autre côté. Et 18 sorties terminées avant le 10 janvier ne sont reçues que pendant la semaine.

Quittez `psql` par `\q`.

Étape 17 — Versionner

Depuis `01_de` :

```
git add flux/pont.py flux/vers_series.py flux/fenetres.py flux/alerte.py
git add tests/test_flux.py sql/temps_reel.sql
git commit -m "Cours 09 : le pont vers Kafka, la base du temps reel et l'alerte de vibration"
```


© 2026 iAlexMG. Tous droits réservés.

Kafka peut s'arrêter : `docker compose stop kafka mqtt` libère près de 300 Mo de mémoire, et ses sujets restent dans son volume. Le Cours 12 les rallumera.

Récapitulatif

Ce que vous avez mesuré	Le chiffre
Une journée de l'usine rejouée	4 048 points en 24 s, égaux à l'historien sans ses 58 valeurs de départ
La semaine de télémétrie	3 449 messages, 70 doublons, 246 reçus plus d'une heure après la fin
L'archive contre l'abonné	3 449 lignes de part et d'autre, 0 écart de contenu dans les deux sens
Les messages retenus	58, un par étiquette, livrés à l'abonnement
La file d'un absent, au réglage par défaut	999 messages retrouvés sur 2 118
Octobre par Kafka	62 726 points relayés et insérés, 0 perdu, 0 en retard sur le filigrane
L'alerte	levée le 22 octobre à 8 h 15, 6 jours 2 h 29 min avant le déclenchement
L'arrêt	levé le 27 octobre à 6 h 15, 1 jour 4 h 29 min avant
La relecture au seuil de 4,0	levée le 21 octobre à 6 h 45
La télémétrie par Kafka	3 449 lus, 3 379 insérés, 70 doublons écartés par la base
Les retards	241 sorties au-delà d'un filigrane d'une heure, jusqu'à 4 jours de retard

Ce que vous savez maintenant et que vous ne saviez pas : l'archive que vous chargez depuis le Cours 03 est ce qu'un abonné écrit ; un flux livre au moins une fois, et c'est la base qui rend l'effet unique ; un journal qu'on garde se relit à volonté, pour poser une question que personne n'avait prévue ; et une alerte qui juge le temps des événements, pondéré et avec une bande morte, voit une panne six jours avant qu'elle arrive.

Dépannage

Symptôme	Cause	Ce qu'on fait
`Le courtier ne répond pas sur localhost:1883 ([WinError 10061] …)`	le service `mqtt` n'est pas démarré, ou Ubuntu s'est arrêté	dans Ubuntu, `docker compose up -d mqtt`, et garder un terminal ouvert

© 2026 iAlexMG. Tous droits réservés.

Symptôme	Cause	Ce qu'on fait
`ModuleNotFoundError: No module named 'paho'` (ou `confluent_kafka`, `psycopg`)	les paquets du chemin chaud ne sont pas installés dans `formations`	`pip install -r ..\01_Data_Engineering\requirements.txt`, depuis `Projet_Boreal_intermediaire`
L'abonné ou le pont ne s'arrête pas seul	il n'a reçu que des messages retenus, qui ne font pas courir le silence	Ctrl+C ; un programme lancé avant le rejeu attend ainsi son premier message neuf
`ModuleNotFoundError: No module named 'flux'`	le programme n'est pas lancé depuis `01_de`, ou le fichier `flux\__init__.py` manque	`cd` vers `Projet_Boreal_intermediaire\01_de`, puis `python -m flux.abonne`
L'abonné compte bien plus de messages que le rejeu n'en publie	sa session persistante a gardé ce qui s'est publié pendant son absence	fermer la session par `python -m flux.abonne --session-propre`, puis Ctrl+C ; effacer `flux\telemetrie`, recommencer
La comparaison de l'étape 5 rend des écarts dans les deux sens	les fichiers de l'abonné contiennent deux rejeux, parce qu'il écrit à la suite	`rmdir /s /q ..\flux\telemetrie` depuis `01_de`, puis rejouer une seule fois
`%4|…|GETPID|…: Failed to acquire idempotence PID from broker … Coordinator load in progress: retrying`	Kafka prépare son coordinateur au premier producteur idempotent après son démarrage	rien : le producteur réessaie seul, et le message ne revient pas ensuite
Le pont écrit, au bout de 30 s, `Kafka refuse un message de usine.points : KafkaError{code=UNKNOWN_TOPIC_OR_PART,…}`	le sujet n'existe pas : la plateforme interdit leur création automatique	refaire l'étape 9, puis relancer le pont, qui recevra de nouveau les messages non acquittés
`KeyError: "SERIES_MOT_DE_PASSE n'est ni dans l'environnement ni dans …"`	le `.env` de `01_de` n'a pas le mot de passe de la base	l'ajouter, comme au début de la seconde séance
`vers_series` refuse de se connecter, erreur d'authentification de psycopg	le mot de passe du `.env` de `01_de` diffère de celui de la plateforme	recopier celui de `plateforme\.env`
L'alerte ne se lève pas, et `temps_reel.points` est vide	le rejeu a été lancé avant le pont : le courtier ne garde rien pour un pont qu'il n'a jamais vu	relancer le pont, puis le rejeu
Le conteneur a disparu entre deux commandes	le dernier terminal Ubuntu s'est fermé, et WSL a arrêté la distribution	rouvrir Ubuntu, `docker compose start mqtt kafka series`, et garder un terminal ouvert

© 2026 iAlexMG. Tous droits réservés.